SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0611 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning ***(BL 5)***

Assessment Tool Weightage: 40%

QUESTIONS: 2 **Total Marks:20**

Annexure A

Coding Challenge

Code of Conduct:

I K.ASWINI (192312397) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

9. Create a Python program to implement the **K-Means clustering algorithm**. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

ANSWERS

8.PYTHON CODE :

```
import math
from collections import Counter
def knn(td, tl, tp, k):
d = [(math.sqrt(sum((a-b)**2 for a,b in zip(x,tp))), y) for x,y in zip(td,tl)]
 return Counter([y for _,y in sorted(d)[:k]]).most_common(1)[0][0]
def acc(a, p):
return sum(i==j for i,j in zip(a,p)) / len(a) * 100
td = [[2,4],[4,4],[4,6],[6,2],[6,4],[6,6]]
tl = ['A','A','A','B','B','B']
test = [[3,5],[5,5]]
actual = ['A','B']
k = int(input("Enter K: "))
pred = [knn(td, tl, t, k) for t in test]
for t, r in zip(test, pred):
print(f"{t} -> {r}")
print(f"Accuracy: {acc(actual, pred):.2f}%")
```

INPUT : ENTER K : **3**
OUTPUT:
[3, 5] -> A
[5, 5] -> A
Accuracy: 50.00%

9.PYTHON CODE

```
import numpy as np
import matplotlib.pyplot as plt
def kmeans(X, k, max_iters=100):
 centroids = X[np.random.choice(len(X), k, replace=False)]
for _ in range(max_iters):
```

```
distances = np.linalg.norm(X[:, None] - centroids, axis=2)
labels = np.argmin(distances, axis=1)
new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])
if np.allclose(centroids, new_centroids, equal_nan=True): # Added equal_nan=True for
stability with nan centroids
break
centroids = new_centroids
 return labels, centroids
n = int(input("Enter number of data points: "))
X = []
for i in range(n):
point = list(map(float, input(f"Enter point {i+1}: ").split()))
X.append(point)
X = np.array(X)
k = int(input("Enter number of clusters (K): "))
if k > len(X):
   print(f"Warning: The number of clusters (K={k}) cannot be greater than the number of data
points (n={len(X)}).")
   print(f"Adjusting K to {len(X)} to proceed.")
   k = len(X)
if k == 0:
   print("Warning: Number of clusters (K) cannot be zero. Adjusting K to 1.")
   k = 1
labels, centroids = kmeans(X, k)
for i in range(k):
   print(f"\nCluster {i+1}:")
     if np.isnan(centroids[i]).any():
       print("  (Empty cluster, centroid is NaN)")
   for j in range(len(X)):
       if labels[j] == i:
           print(X[j])
print("\nCentroids:\n", centroids)
plt.figure(figsize=(8, 6))
if X.shape[1] == 1: # 1-dimensional data
   plt.scatter(X[:,0], np.zeros_like(X[:,0]), c=labels, cmap='viridis', label='Data Points')
    valid_centroids = centroids[~np.isnan(centroids).any(axis=1)]
   if valid_centroids.size > 0:
       plt.scatter(valid_centroids[:,0], np.zeros_like(valid_centroids[:,0]), marker='X', s=200,
color='red', label='Centroids')
   plt.yticks([]) # Hide y-axis ticks for 1D visualization
   plt.xlabel('Feature 1')
elif X.shape[1] >= 2: # 2-dimensional or higher-dimensional data (plotting first two
dimensions)
   plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis', label='Data Points')
   valid_centroids = centroids[~np.isnan(centroids).any(axis=1)]
   if valid_centroids.size > 0:
       plt.scatter(valid_centroids[:,0], valid_centroids[:,1], marker='X', s=200, color='red',
label='Centroids')
   plt.xlabel('Feature 1')
   plt.ylabel('Feature 2')
```

```
else:
    print("Cannot plot data with 0 dimensions.")
plt.title("K-Means Clustering")
plt.legend()
plt.grid(True)
plt.show()
```

INPUT :
Enter number of data points: 5
Enter point 1: 2
Enter point 2: 3
Enter point 3: 1
Enter point 4: 5
Enter point 5: 4
Enter number of clusters (K): 2

OUTPUT:
Cluster 1:
[2.]
[3.]
[1.]

Cluster 2:
[5.]
[4.]

Centroids:
[[2.]
[4.5]]